Real-Time Arrhythmia Detection on a Microcontroller — Project Roadmap

Goal: A single-lead ECG pipeline that filters the signal, detects R-peaks, and classifies rhythm, running in real time on an STM32 (Cortex-M4F), with measured latency under 50 ms. Phase 2 swaps the classical classifier for a quantized neural net and adds a wearable form factor.

> **Not a medical device.** This is an engineering portfolio project. Nothing here is for diagnosis or clinical use. State this in the README and in any demo.

Guiding principle: software-first

Build and validate the whole algorithm on your laptop against reference data **before** the hardware arrives, then port it. Face real-world electrical noise last. This is both faster and the correct engineering order.

Order of difficulty (and the order we tackle it): 1. Clean reference data (MIT-BIH) on a laptop — easy, provable. 2. Same algorithm in portable C on a laptop — de-risks the port. 3. Same C on the STM32, fed recorded data — proves real-time timing. 4. Live ECG from the AD8232 — the messy part (noise, motion, 60 Hz mains).

Bill of materials

Item	Choice	~Cost	Why
MCU board	STM32 **Nucleo-F411RE**	$15	Cortex-M4F: FPU + DSP instructions needed for CMSIS-NN in Phase 2
ECG front-end	**AD8232** module (w/ electrodes + leads)	$12	Analog out straight into ADC; built-in 0.5–40 Hz filtering, gain ~1100
Electrode pads	Disposable Ag/AgCl (3-pack lead)	$8	Consumable; buy extras
USB cable	Mini/Micro-USB (check board)	—	Nucleo has onboard ST-Link, no separate programmer needed
Phase 2 only	STM32 **Nucleo-L4** (e.g. L432KC)	$12	Ultra-low-power modes for the "wearable + power measurement" story

Everything except the L4 is Phase 1. The Nucleo has an onboard debugger — you do **not** need a separate ST-Link or J-Link.

Toolchain

- **STM32CubeIDE** (free, ST's official Eclipse-based IDE) — drivers, code generation, flashing, debugging in one install.
- **Python 3.11**, packages: `wfdb`, `numpy`, `scipy`, `matplotlib`, `scikit-learn`. Phase 2 adds `tensorflow`.
- Serial dashboard: your Next.js skills apply — a **Web Serial API** dashboard in the browser is a natural, impressive choice. A quick `pyserial` + matplotlib script is the fast fallback.

Naming note on the ML runtime: Google rebranded *TensorFlow Lite Micro (TFLM)* to *LiteRT for Microcontrollers* in 2024. Same thing. The GitHub repo is still `tensorflow/tflite-micro` and the C++ API is still `tflite::` — so tutorials under both names are valid. Don't let the two names confuse you.

PHASE 1 — Classical pipeline, real-time on hardware (Months 1–3)

Stage 1.1 — Offline pipeline in Python (Weeks 1–3)

Prove the algorithm on clean data first.

1. Download MIT-BIH Arrhythmia Database via `wfdb.rdrecord` / `wfdb.rdann` (360 Hz, 48 records, expert beat annotations).
2. **Bandpass filter** the raw signal ~5–15 Hz (this band maximizes QRS energy — the Pan-Tompkins choice).
3. **Pan-Tompkins R-peak detection:** derivative → squaring → moving-window integration → adaptive thresholds. Implement it yourself.
4. **Beat classification:** start simple. Group MIT-BIH's per-beat labels into the 5 **AAMI classes** (N, S, V, F, Q). A first classifier using RR-interval features + basic QRS morphology can separate Normal from PVC well.
5. **Validate properly:** - R-peak detection: report **sensitivity** and **positive predictive value (PPV)** against annotations. - Classification: confusion matrix + per-class precision/recall. - **Use the inter-patient split** (de Chazal: train on one set of patients, test on *different* patients). Most amateur projects leak the same patient into train and test and report fake-high accuracy. Doing the inter-patient split correctly is a real differentiator and signals you understand the field.

Checkpoint 1.1: R-peak sensitivity > 99% on record 100; a confusion matrix over the AAMI classes on held-out patients; plots of detected peaks over the raw waveform.

Stage 1.2 — Reimplement in portable C on the laptop (Weeks 3–5)

1. Rewrite the pipeline in plain C (no STM32 headers yet) — compile and run on your PC.
2. Feed it the same MIT-BIH records (export them to a CSV/array).
3. **Bit-compare** C output against the Python reference. They should agree.

4. Decide **fixed-point vs float**. The F411 has an FPU, so `float` is fine to start; note fixed-point as a possible optimization later.

Checkpoint 1.2: C pipeline reproduces the Python R-peaks and labels on the same records.

Stage 1.3 — Get it onto the STM32, fed recorded data (Weeks 5–9)

1. **Blinky + UART "hello world"** first. Confirm you can build, flash, and see serial output. Don't skip this.
2. Load a MIT-BIH record into the MCU — either stored as a `const` array in flash, or streamed from the PC over UART. This lets you test the real-time pipeline on hardware *before* you have a live signal.
3. Drive processing from a **hardware timer** at the record's sample rate (360 Hz). Use a **ring buffer** so acquisition and processing don't collide.
4. **Measure latency** with the DWT cycle counter (or toggle a GPIO pin and watch it on a cheap logic analyzer). Time from "sample in" to "classification out."

Checkpoint 1.3: On-device classifications match your desktop C on the same record; measured end-to-end latency logged (target < 50 ms — you'll likely be far under).

Stage 1.4 — Live ECG (Weeks 9–12)

1. Wire **AD8232 OUT → STM32 ADC**. Use **ADC + DMA triggered by a timer** for jitter-free sampling. Wire LO+/LO− (lead-off detect) to GPIOs.
2. Handle real-world noise the recorded data didn't have: - **Baseline wander** (breathing) — highpass / detrend. - **60 Hz mains hum** (Canada is 60 Hz) — notch filter. - Motion artifacts — the AD8232 helps, but expect noise.
3. Stream classifications + heart rate to your dashboard.

Checkpoint 1.4 (Phase 1 done): Live single-lead ECG, real-time R-peak + rhythm/HR on a dashboard, latency documented. **This alone is a strong, complete portfolio project.**

PHASE 2 — Neural net on-device + wearable (Months 4–12)

Stage 2.1 — Train a small 1D-CNN (Weeks 13–16)

- In Keras/TensorFlow, train a compact 1D-CNN on segmented MIT-BIH beats → AAMI classes.
- Keep it tiny (tens of KB target). Keep the same inter-patient split.

Stage 2.2 — Quantize + convert (Weeks 16–18)

- **Int8 post-training quantization** via the LiteRT converter → `.tflite` → C byte array.
- Compare quantized vs float accuracy (expect a small drop).

Stage 2.3 — Deploy with TFLM/LiteRT + CMSIS-NN (Weeks 18–24)

- Integrate the LiteRT-for-Microcontrollers runtime with **CMSIS-NN** kernels (they exploit the M4's DSP instructions — this is why we chose the F411).
- Run inference on-device. Log **flash + RAM (tensor arena) usage**.
- **Head-to-head:** classical vs NN — accuracy, latency, footprint. This comparison table is the centerpiece of the writeup.

Stage 2.4 — Wearable + rigorous evaluation (Weeks 24–52)

- Move to **STM32L4** for low-power modes; measure current draw in sleep vs active.
- **3D-printed enclosure**, battery, real electrode placement.
- Final evaluation doc: latency, **power consumption**, **false-positive rate**, per-class sensitivity/PPV, memory footprint. Numbers, tables, plots.

Checkpoint Phase 2: Wearable device running a quantized NN in real time, with a documented classical-vs-NN comparison and a power/latency/accuracy report.

Suggested repo structure

```
arrhythmia-detection-mcu/
├── README.md                # what it is, demo GIF, results table, "not a medical device"
├── ROADMAP.md               # this file
├── LICENSE
├── python/
│   ├── pipeline/            # Stage 1.1: filters, pan_tompkins, classifier
│   ├── train_cnn.py         # Stage 2.1
│   ├── quantize_convert.py  # Stage 2.2
│   └── notebooks/           # validation, confusion matrices, plots
├── c-reference/             # Stage 1.2: portable C, runs on PC, tested vs Python
│   └── tests/
├── firmware/                # STM32CubeIDE project
│   ├── Core/
│   ├── model/               # Phase 2: model.cc byte array + TFLM/CMSIS-NN
│   └── README.md            # wiring diagram, build/flash steps
├── dashboard/               # Web Serial (Next.js) or pyserial script
└── docs/
```

```
    ├── results.md            # latency, power, FP rate, per-class metrics
    └── wiring.png
```

Commit continuously with a clean history (you already use feature-branch PR workflow — do that here too). Each checkpoint is a natural PR + release tag.

Kardium interview talking points (keep a running notes file as you build)

- **R-wave detection:** how Pan-Tompkins works and why the 5–15 Hz band maximizes QRS energy.
- **Filter cutoffs:** why 0.5 Hz highpass (baseline wander) and 40 Hz lowpass (noise) for monitoring, and how that trades off against a wider diagnostic band.
- **Signal-to-noise tradeoffs:** RLD/common-mode rejection, 60 Hz notch, motion artifacts.
- **Real-time on embedded:** ring buffer, timer-driven ADC+DMA, measured <50 ms latency — vs. the usual batch-on-a-laptop project.
- **Inter-patient evaluation:** why patient-independent splits matter and how naive splits inflate accuracy.
- **Single-lead limits:** AD8232 is single-lead; multi-lead needs something like an ADS1298 (good "future work" line).

Your differentiators (say these explicitly in the README)

1. **Real-time on hardware**, not batch on a laptop — with a measured latency number.
2. **Classical vs quantized-NN comparison** on the same device — accuracy, latency, flash/RAM, power.
3. **Correct inter-patient evaluation** — you understand the methodology, not just the API.